

Week 5 Reflection

Anomaly Detection + Agentic RCA with OpenTelemetry — CSE636 DevOps for AI

What worked

The full pipeline — detect, group, investigate, report, trace — ran end to end on the first real attempt without silent failures. IsolationForest at contamination=0.04, tuned during the Lab's sweep, correctly separated the one real 16-point incident (INC-002: CPU peaked 91.56%, latency peaked 3661.8ms) from four single-point false positives in the Assignment's grouping stage too, which confirmed the Lab and Assignment stages compose correctly rather than just working in isolation.

DBSCAN comparison (Lab Part A bonus)

The optional bonus asked whether DBSCAN or IsolationForest performs better on this dataset, and why. The first DBSCAN attempt (eps=0.9, min_samples=5) underperformed IsolationForest — recall dropped to 0.62 because 6 of the 16 incident minutes, sitting at the edges of the spike with softer values, formed their own small dense sub-cluster instead of being flagged as noise.

Sweeping eps against the same standardized features told a clearer story:

eps	Flagged	Precision	Recall	F1
0.3	18	0.89	1.00	0.94
0.5	16	1.00	1.00	1.00
0.7	11	1.00	0.69	0.81
0.9	10	1.00	0.62	0.77
1.2	1	1.00	0.06	0.12

eps=0.5 produced a perfect detector — precision, recall, and F1 all 1.00 — actually beating IsolationForest's best F1 of 0.89 at contamination=0.04. A follow-up sweep of min_samples (3, 5, 8, 10, 15, 20, 100, 200, 300) at eps=0.5 showed the result is flat and perfect through min_samples=20, then breaks down starting at 100 (precision drops to 0.89 as normal points near the edge of the main cluster stop meeting the density bar) and collapses further at 200/300.

Conclusion: DBSCAN's ceiling is higher on this dataset once tuned, because the injected incident genuinely is a tight spatial cluster in standardized feature space — exactly the structure DBSCAN is built to separate from noise. But eps has a narrow “just right” zone (0.5 works well; 0.7 already drops F1 to 0.81), while IsolationForest's contamination parameter is more forgiving of a rough guess. min_samples, by contrast, barely matters once eps is correct — robust across a 3–20 range before degrading. The pipeline in src/anomaly_detector.py still uses IsolationForest for this write-up's contamination framing; DBSCAN (eps=0.5, min_samples=5) is a documented, testable improvement not yet swapped into the running pipeline.

What broke

I did not hit a build failure of my own during the Assignment's plumbing — telemetry.py's exporter already included the `os.makedirs(...)` guard against writing to a not-yet-created output/ directory (a real `FileNotFoundError` that surfaces if that line is removed, since `ConsoleSpanExporter` tries to open the target file immediately). The real iteration and debugging in this week's work happened in the Lab's tuning sweeps: catching the accuracy trap at `contamination=0.01` (98% accuracy while missing 75% of the real incident), and diagnosing why the first DBSCAN attempt undercounted recall before finding `eps=0.5`.

Reproduced this directly to confirm the claim rather than just asserting it: temporarily commented out the `os.makedirs(...)` line in `telemetry.py`, deleted `output/`, and reran `rca_agent.py`. This produced the exact predicted failure — `FileNotFoundError: [Errno 2] No such file or directory: '../output/spans_sample.json'` — at `telemetry.py`'s `open(export_path, "a")` call. Restoring the line and rerunning regenerated `output/` cleanly with all 5 reports and `spans_sample.json`, confirming the fix genuinely resolves it.

What I'd do differently with more time

- Swap the Assignment's detector from `IsolationForest` to `DBSCAN` (`eps=0.5`, `min_samples=5`) now that the Lab bonus shows it outperforms `IsolationForest` here (F1 1.00 vs 0.89) — currently a documented alternative rather than the detector actually running in `src/anomaly_detector.py`.
- Wire in a real `ANTHROPIC_API_KEY` and compare the simulated threshold-based RCA (“if CPU > 80%, blame CPU saturation”) against actual model reasoning on the same incident.
- Alert grouping is purely time-based right now; a real system would also correlate against deploy events or config changes in the same window, which this synthetic dataset doesn't model.

Honest limitation

The RCA reports are threshold logic, not AI reasoning — a reasonable first pass, but it would not generalize to incident types this dataset doesn't model (a memory leak with normal CPU, or a slow downstream dependency with no metrics signature of its own). Every generated report is honestly labeled “confidence: low (simulated response)” rather than presented as if it came from a real call.